

StSATO vs EGGS Finance Contract Differences

Prepared by: Infinite Trading

Purpose: side by side comparison of the EGGS Finance contract and the deployed StSATO contract

Executive Summary

StSATO is a fork of the **EGGS** contract from EGGS Finance, adapted for the SATO token on Ethereum mainnet. The core architecture remains the same: bonding curve pricing, fee accumulation into backing, collateralised borrowing, liquidation by day buckets, and a monotonic price invariant enforced on chain.

The differences are not cosmetic. They are the changes required to make the protocol safe and functional for SATO, to remove deployer privilege, and to preserve the economic intent of the original system while matching the SATO token interface.

Side by Side Comparison

Area	EGGS Finance Contract	StSATO
Backing token	EGGS	SATO
Token name and symbol	EGGS branded token	stsato
Network	EGGS deployment chain	Ethereum mainnet
Bootstrap recipient	Not permanently fixed to 0xdead	All bootstrap stSATO minted to 0xdead
Ownership after launch	Not documented here	Permanently renounced in setStart()
Fee burn mechanism	Calls token burn interface on backing token	Transfers fee burn share to 0x00000000000000000000000000000000dEaD
lastPrice after bootstrap	Not explicitly set	Explicitly set to 1e18
lastLiquidationDate initialization	Different deployment context	Initialized to next midnight at deployment
Frontend state checks	Original EGGS assumptions	Explicit guidance added for SATO loan state and liquidation timing
Audit basis	Cantina report for EGGS	Infinite Trading review plus EGGS inheritance and SATO-specific fixes

Contract Differences

1. Backing Asset

The original contract was built around EGGS. StSATO replaces the backing asset with SATO at address `0x829f4B62EEBE12Af653b4dD4fFc480966F7d7f09`.

This is the most important change because it changes the token interface, the token economics, and the burn mechanism.

2. Fee Burn Path

EGGS used a backing token burn call. StSATO does not use the same selector, because SATO exposes a different burn function and restricts it to the minter role.

Item	EGGS	StSATO
Fee burn call	<code>burn(amount)</code> style interface on the backing token	<code>safeTransfer(DEAD, burnAmount)</code>
Risk	Works for the original token assumptions	Direct selector mismatch would revert every fee collection path
Result	Deflation through token burn	Deflation through transfer to dead address

This is the critical protocol fix. Without it, buys, sells, borrow operations, leverage positions, and loan maintenance would revert.

3. Bootstrap Design

StSATO makes the bootstrap explicitly fair launch.

Item	EGGS	StSATO
Bootstrap mint	Original deployment pattern	All bootstrap stSATO goes to <code>0xdead</code>
Deployer allocation	Not the same invariant	None
Initial price	Implicit in original deployment	Explicitly locked at 1 SATO per stSATO

This removes deployer advantage and makes the first market price consistent and auditable.

4. Price Initialization

EGGS did not carry over the same explicit `lastPrice` initialization path. In StSATO, `lastPrice` is set to `1e18` inside `setStart()`.

Item	EGGS	StSATO
<code>lastPrice</code> after bootstrap	Not set in the same way	Set to <code>1e18</code>
Safety check behavior	First trade could pass trivially from zero	First trade is measured against the bootstrap price

5. Ownership Model

StSATO renounces ownership atomically in `setStart()` .

Item	EGGS	StSATO
Admin control after launch	Depends on original deployment model	None
Upgradeability	Not part of this contract	Not upgradeable
Ownership after bootstrap	Not locked here	<code>address(0)</code> permanently

6. Liquidation Timing

StSATO initializes `lastLiquidationDate` from the deployment timestamp rounded to the next midnight.

Item	EGGS	StSATO
<code>lastLiquidationDate</code> initial value	Original deployment context	Next midnight after deployment
Purpose	Start day bucket processing	Avoid epoch zero backlog and keep liquidation bounded in practice

7. Fee Parameters

The core fee structure remains economically similar, but the StSATO document and deployment should be read in the context of SATO and the updated fee routing.

Parameter	Value in StSATO
<code>buy_fee</code>	990
<code>sell_fee</code>	990
<code>buy_fee_leverage</code>	10
Trade fee routing	Fee share stays in backing, small share is transferred to <code>0xdead</code>

8. Frontend and Integration Behavior

The contract itself is correct, but the SATO deployment surfaced two integration rules that must be followed by the frontend.

Topic	Required behavior
Loan existence	Check <code>borrowed > 0</code> from <code>getLoanByAddress()</code>
Loan expiry	Treat <code>isLoanExpired()</code> as an expiry check, not a loan existence check
Liquidation backlog	Guard against <code>lastLiquidationDate > block.timestamp</code> before computing <code>daysBehind</code>
Bounded recovery	Use <code>drainLiquidations(maxDays)</code> when needed

Specific Fixes Made for StSATO

Fix	Why it was needed	Outcome
Fee burn transfer to dead address	The SATO burn interface is not compatible with the original EGGS pattern	Trading and lending remain functional
<code>lastPrice = 1e18</code> in <code>setStart()</code>	Prevents a zero price bootstrap state	Price invariant starts from the correct baseline
Bootstrap mint to <code>0xdead</code>	Ensures no deployer allocation	Fair launch structure preserved
Ownership renounced inside <code>setStart()</code>	Removes admin control after launch	Contract becomes immutable after bootstrap
<code>lastLiquidationDate</code> set at deployment	Prevents the liquidation loop from starting at epoch zero	Liquidation remains bounded and practical

Inherited Design From EGGS

StSATO preserves the core design of the EGGS contract:

Mechanism	Preserved in StSATO
Bonding curve pricing	Yes
On chain price monotonicity check	Yes
Borrow and repay lifecycle	Yes
Leverage positions	Yes
Day bucket liquidation accounting	Yes
No oracle dependency	Yes
No upgrade path	Yes

This is important because the security properties proven in the original audited design still apply, provided the SATO specific interface differences are handled correctly.

Security Impact Summary

The StSATO changes fall into three categories.

Category	Meaning	Examples
Compatibility fixes	Required to make the EGGS design work with SATO	Fee burn transfer, token address substitution

Safety fixes	Remove bootstrap or invariant risk	lastPrice = 1e18, ownership renunciation
Integration fixes	Required for correct frontend or operational behavior	isLoanExpired interpretation, liquidation timing guard

No change introduces a privileged backdoor or upgrade mechanism. The protocol remains trustless after bootstrap.

Conclusion

StSATO is not a cosmetic rename of EGGS. It is the EGGS protocol design adapted to a different backing asset, a different chain, and a stricter launch model.

The major differences are the SATO specific fee burn path, fair launch bootstrap, explicit price initialization, permanent ownership renunciation, and deployment time liquidation initialization. These changes are the reason the contract works on Ethereum mainnet with SATO and remains auditable as an immutable protocol.